# OPERATING SYSTEMS – ASSIGNMENT 1

**Name:** S. Sharvesh **Register No:** 192525458

## TASK 7 – Evaluate Process Control System Calls (4 Marks)

```
Evaluate fork(), exec(), wait() and exit(); how a shell runs a command and obtains exit status; zombie and orphan processes
Complete the Code: Use fork(), execlp() and waitpid(). The child runs expr 6 * 7 and exits 127 if exec fails; the second ch
```

Expected output:

shell: running job 1

42

job 1 exit status 0

shell: running job 2

job 2 exit status 3

```
Explain why stdout is flushed before each fork(), and why removing the second fflush and changing _exit(3) to exit(3) can d
Alternative Concept and Complexity Analysis: Compare posix_spawn() or threads with fork()+exec() for 1,000 commands, memory
```

Evidence Expected: Working program, process diagram and reasoned evaluation.

### ANSWER

7.1 Process-control calls

7.2 How a shell executes a command

The shell reads and parses the command line, expands arguments and checks redirections.

```
The shell calls fork() to create a child process.
```

The child configures redirections and pipes, then calls an exec-family function to replace itself with the requested command.

```
The parent shell calls waitpid() for a foreground command; for background commands it records the PID and collects completi
When the child terminates, the kernel stores its termination status. waitpid() returns that status to the parent.
```

The shell checks WIFEXITED(status) and, if true, obtains the code using WEXITSTATUS(status). It makes the status available as the shell's command status.

```
A zombie is a terminated child whose parent has not yet collected its exit status. The kernel retains a small process-table
fork() followed by exec() separates process creation from program loading. The shell can create a child, set up pipes, file
```

7.3 Complete Code – TODO 1

#define _POSIX_C_SOURCE 200809L

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void)
{
printf("shell: running job 1\n");
```

fflush(stdout);

```
pid_t pid = fork();
```

if (pid < 0) {

perror("fork");

```
return 1;
```

```
  }
```
if (pid == 0) {
```
  execlp("expr", "expr", "6", "*", "7", (char *)NULL);
```
perror("execlp");

_exit(127);
```
  }
```
int status;
```
  if (waitpid(pid, &status, 0) < 0) {
```
perror("waitpid");
```
  return 1;
  }
```
if (WIFEXITED(status))
```
  printf("job 1 exit status %d\n", WEXITSTATUS(status));
```
else
```
  printf("job 1 did not exit normally\n");
  printf("shell: running job 2\n");
```
fflush(stdout);
```
  pid = fork();
```
if (pid < 0) {

perror("fork");
```
  return 1;
  }
```
if (pid == 0) _exit(3);
```
  if (waitpid(pid, &status, 0) < 0) {
```
perror("waitpid");
```
  return 1;
  }
```
if (WIFEXITED(status))
```
  printf("job 2 exit status %d\n", WEXITSTATUS(status));
  return 0;
  }
```
7.4 Compile and expected output

gcc -std=c11 -Wall -Wextra process_demo.c -o process_demo

./process_demo

shell: running job 1

42

job 1 exit status 0

shell: running job 2

job 2 exit status 3

The first fflush(stdout) empties the parent's output buffer before fork. Otherwise, if stdout is redirected to a file, the buffered text is copied into the child’s address space. If the child later calls exit(), both parent and child may flush copies of the same text, making “shell: running job 2” appear twice. The given child uses _exit(3), which does not flush stdio, so the duplication demonstration specifically requires changing _exit(3) to exit(3) and removing the

second fflush as the question instructs.

7.5 Alternative: posix_spawn() or threads

posix_spawn() combines process creation and program launch through a single API and may be more efficient on some systems because the implementation can avoid a full fork-like path. It still creates a separate process with its own address space and strong isolation. Threads share the process address space, file descriptors and many resources; they are cheaper to create in many workloads but have weaker isolation. A faulty command running in a thread can corrupt shared memory or crash the shell process.

```
For 1,000 commands, process creation and execution costs are approximately O(1000) operations; total elapsed time depends o
```

## QUESTION FROM THE PROVIDED ASSIGNMENT

Call | Function

fork() | Creates a child process. The child initially receives a logical copy of the parent's address space, commonly implemented with copy-on-write.

exec() family | Replaces the calling process image with a new program. The PID remains the same; successful exec does not return.

wait()/waitpid() | Allows a parent to collect a child’s termination status and release its process-table resources.

exit()/_exit() | Terminates a process. exit() flushes C stdio streams and runs registered handlers; _exit() terminates without normal stdio flushing.